

STUDER — Project Overview

▼ Relevant source files

Backend/STUDER/Dockerfile

Backend/STUDER/pom.xml

Docker/docker-compose.yml

Frontend/STUDER/PROJECT_DOCUMENTATION.md

Frontend/STUDER/package-lock.json

Frontend/STUDER/package.json

README.md

STUDER is an open collaborative platform designed for the creation, reuse, and evolution of educational knowledge. Inspired by software engineering principles—such as modularity, version control, and open-source collaboration—the platform treats educational content as dynamic "blocks" rather than static documents. These blocks can be forked, adapted, and validated by the community, fostering a sustainable ecosystem where learning and contributing are integrated processes

README.md

15-34

Educational Mission

The project addresses common issues in digital learning environments, such as content duplication, fragmented resources, and poor maintainability

README.md

53-62

STUDER

introduces several core concepts to solve these problems:

Concept	Description
Reusable Educational Blocks	Independent units representing theory, exercises, or multimedia README.md 68-76
Versioning & Forks	Ability to improve existing blocks while preserving authorship history README.md 77-84

Concept	Description
Collaborative Validation	Quality control through community interaction, likes, and adoption README.md 85-93
Social Layer	Gamification via reputation, rankings, and thematic contests README.md 94-103
Sources:	README.md 15-103

System Architecture

STUDER is built using a multi-tier architecture containerized with Docker. It consists of an Angular frontend, a Spring Boot backend, a Python-based media processing service, and supporting infrastructure like PostgreSQL and MinIO.

High-Level Component Interaction

This diagram illustrates how the major subsystems and code entities interact across the network boundaries defined in the infrastructure.

Sources:

Docker/docker-compose.yml

1-94

Frontend/STUDER/PROJECT_DOCUMENTATION.md

17-30

For a deep dive into the technology stack and networking, see [Architecture & Technology Stack](#).

Development Roadmap

The project follows an agile methodology (Scrum) with development organized into iterative sprints

- Sprint 0 — Foundation:** Initial setup of the architecture, internationalization (i18n), and theme systems

README.md

146-156
- Sprint 1 — Social Core:** Implementation of JWT security, social graph (followers), and communication (messaging/discussions)

README.md

159-169

- **Sprint 2 — Content System:** Development of educational blocks and learning paths

README.md 172-178

- **Sprint 3 — Versioning:** Forking mechanics and advanced search/filtering

README.md 181-189

- **Sprint 4 — Community:** Gamification, rankings, and the global activity feed

README.md 191-199

Sources: README.md 134-199

Subsystem Integration

The codebase is split into three primary repositories/directories that collaborate to provide the full functionality.

Code Entity Mapping

This diagram bridges the natural language functional areas to specific code entities and configurations.

Sources: Frontend/STUDER/PROJECT_DOCUMENTATION.md 17-30 Backend/STUDER/pom.xml 1-50

Docker/docker-compose.yml 17-50

Core Subsystems

- **Backend (`app-be`):** A Java 17 / Spring Boot 4 application managing the business logic, PostgreSQL persistence via JPA, and JWT-based security Backend/STUDER/pom.xml 30-75
- **Frontend (`frontend`):** An Angular 19 SPA utilizing PrimeNG for UI components and `@ngx-translate` for multi-language support Frontend/STUDER/package.json 13-24
- **Media Service (`studer-media`):** A Python service that handles image transformations (variants like avatars and thumbnails) and stores them in MinIO

Docker/docker-compose.yml 36-50

To begin setting up the environment, see [Getting Started & Local Development](#).

Detailed Documentation Sections

- [Architecture & Technology Stack](#) — Detailed view of the multi-tier architecture, network topology, and service communication.
- [Getting Started & Local Development](#) — Instructions for local setup using Docker Compose, environment variables, and build tools.